

Way to read CV/ML papers that saves time

Author: Dr Prashant Aparajeya

Date: 19 Sept 2025

0) Quick triage (3–7 min)

- Read: title → abstract → figures/captions → conclusion.
- Answer fast:
 1. What problem & setting? (task, dataset/s, constraints)
 2. What's the *claim*? (accuracy, speed, data/compute savings, robustness, simplicity)
 3. Is the evidence likely enough to justify a deep read?
- Exit early if: unclear setting, tiny gains with huge compute, weak baselines, or missing code/data.

1) First pass (skim, ~10–15 min)

- Map the context: prior SOTA they compare to, where it plugs in (backbone/loss/training, inference).
- Collect the nouns: datasets, metrics, compute, backbones, training length, input resolution.

- **Note the scope:** supervised/unsupervised/self-supervised; closed-set/open-vocab; zero/few-shot; real-time vs offline.
- **Write 3 bullets:** key idea, key evidence, key caveat.

2) Second pass (careful read, ~40–60 min)

Method

- **Reconstruct the algorithm** in your own words or a block diagram. If there's math, rewrite each symbol with a plain-English tag.
- **Find the “pressure points”:** assumptions, hyperparameters that matter, tricks (augmentations, losses, schedulers).
- **Complexity:** FLOPs, params, memory, latency; training vs inference cost.

Experiments

- **Baselines:** Are they strong, recent, and tuned? Same input size, epochs, data?
- **Ablations:** Do they isolate each component's contribution? Any “remove this → collapse” signs?
- **Fairness checks:** Same augmentations, same compute budget, same evaluation protocol?
- **Generalization:** multiple datasets? cross-domain? long-tail performance? calibration?
- **Robustness:** seeds, confidence intervals, corruption tests, distribution shift tests.

3) Third pass (deep dive / reproduce)

- **Replicate the main table:** match data pre-processing, seeds, eval script. Record diffs.
- **Stress tests:** label shuffling (should crash), reduced data (does it degrade gracefully?), hyperparameter sensitivity.
- **Decide adoption:** “Ship”, “Watch”, or “Pass”, with one sentence for each.

What to extract (your “paper dossier”)

- **Problem:** task, setting, constraints.
 - **Contribution:** X (idea) gives Y (benefit) under Z (conditions).
 - **Evidence:** datasets + metrics + headline numbers (with CIs if given).
 - **Cost:** training compute, inference latency/memory.
 - **Limitations:** author-stated + your own.
 - **Reproduction notes:** code link, license, missing bits, gotchas.
 - **Adoption plan:** where it fits in your stack; risks.
-

CV/ML evaluation red flags (trust your Spidey sense)

- **Apples vs oranges:** different input resolutions, train durations, data volume, or test-time aug only for the proposed method.
 - **Weak or stale baselines:** missing strong recent methods or untuned hyperparams.
 - **Metric games:** only reporting a subset (e.g., COCO AP@0.5 but not AP@[.5:.95]); tiny absolute gains within noise.
 - **Data leakage:** pretraining or prompt curation that overlaps test; using val as test.
 - **Heavy hand-tuning:** "hero" hyperparameters that don't transfer; brittle tricks.
 - **Missing reproducibility:** no seeds, no code, no details for data filtering or augmentations.
 - **Compute asymmetry:** method trains 4× longer or with larger backbones than baselines.
 - **Cherry-picked visuals** without quantitative support.
-

Reading different paper types

- **Algorithm/architecture:** focus on ablations, complexity, and fairness of comparisons.
 - **Dataset/benchmark:** sample collection, licensing, demographics, annotation quality, splits, intended use & limits.
 - **Theory:** understand assumptions; check if the empirical setup matches them. Translate lemmas → intuitions.
 - **Survey:** build a map of paradigms + trade-offs; mine references for what to read next.
-

Math & figures, fast

- **Equations:** relabel symbols (e.g., λ = weight on reg loss), check limiting cases ($\lambda \rightarrow 0$, $\tau \rightarrow \infty$).
- **Diagrams:** redraw the pipeline: inputs $\rightarrow$ modules $\rightarrow$ losses $\rightarrow$ outputs.
- **Tables:** highlight rows where protocol truly matches; note bold vs statistically meaningful.

Reproduction checklist (copy/paste for your notes)

- ☐ Code & commit/tag; license compatible with your use
- ☐ Exact **data** versions + pre-processing + filters
- ☐ **Seeds** and #runs; report mean $\pm$ std / CI
- ☐ **Compute:** GPU/TPU type, hours, batch sizes, mixed precision
- ☐ **Eval script** matches paper; no hidden test-time aug
- ☐ **Ablations** recreate headline gains
- ☐ **Sensitivity** to key h-params; early-exit behaviour
- ☐ **Transfer** to at least one new dataset/domain

One-page template (use for every paper)

Title & Link: Problem & Setting: Core Idea (2–3 sentences): Main Claims (bullets): Key Figure/Table: Datasets & Metrics: Results (with baselines, same protocol): Compute (train/infer), Params, FLOPs: Ablations worth trusting (why): Limitations (theirs & yours): Repro status (matched? diffs?): Ad option plan (where/why/risks): Open questions / follow-ups:

Pro tips

- **Read backward:** conclusions $\rightarrow$ experiments $\rightarrow$ method $\rightarrow$ intro. Hype dilutes as you go.
- **Translate to code:** if you can't stub the forward pass in 30–60 lines, you might not truly understand it.

- **Track deltas, not absolutes:** is the *incremental* gain worth the added complexity and cost?
- **Compare per-unit compute:** accuracy per FLOP or per GPU-hour often changes the ranking.
- **Build a living baseline:** keep your own strong baseline pipeline to test new ideas quickly.
- **Archive summaries:** 5–8 sentence write-ups make future you very happy.

Example

Paper is **Noise2Noise: Learning Image Restoration without Clean Data** (ICML 2018). Here's the framework applied to it:

Quick triage (3–7 min)

- **Problem & setting:**
 - Train denoisers/reconstructors **without clean targets** by pairing *independent corrupted* observations of the same signal.
 - Works for Gaussian/Poisson/Bernoulli noise, Monte-Carlo render denoising, and under-sampled MRI.
- **Core claim:**
 - With the right loss (e.g., L2 for zero-mean noise), training on noisy targets reaches **parity with clean-target training**; sometimes slightly better.
- **Evidence snapshot:**
 - Table 1 shows near-identical PSNR for clean vs noisy targets across Kodak/BSD300/Set14; e.g., Gaussian $\sigma=25$ averages **31.63 dB (clean) vs 31.61 dB (noisy)**; Poisson **30.59 vs 30.57**; Bernoulli **31.85 vs 32.02**.

One-page dossier

Title & venue: Noise2Noise: Learning Image Restoration without Clean Data (ICML 2018).

Problem & setting: Learn f_{θ} mapping corrupted $\rightarrow$ clean using only corrupted pairs $(\hat{x}, \hat{y})$ drawn independently from the same underlying signal.

Core idea (2–3 sentences): For losses that estimate an expectation (L2) or median (L1), the network's optimum is unchanged if you replace clean targets with **independent noisy targets** whose expectation/median equals the clean signal. Therefore, you can train from noisy-noisy pairs and still learn the clean signal.

Main claims:

- Noisy-target training **matches** clean-target training for i.i.d. noise; convergence is similar.
- Extends beyond denoising to **MRI reconstruction** from random undersampling with comparable PSNR.
- In Monte-Carlo rendering, noisy targets are **$\sim 2000\times$ faster to generate** while reaching near-clean quality.

Key figure/table: Fig. 1 (clean vs noisy training curves + capture-budget study). Table 1 (PSNR parity across datasets/noise types).

Datasets & metrics: Train on ImageNet (50k) crops 256×256 with randomized $\sigma \in [0, 50]$; evaluate on Kodak, BSD300, Set14; metric PSNR.

Results (same protocol):

- **Gaussian $\sigma=25$:** 31.63 (clean) vs 31.61 (noisy); BM3D ~ 0.7 dB worse.
- **Poisson $\lambda=30$:** 30.59 vs 30.57; (inverse Anscombe) 28.36.
- **Bernoulli $p=0.5$:** 31.85 vs 32.02; (DIP) 30.14.
- **MRI (10% k-space):** Example: input 18.93 dB $\rightarrow$ 29.77 dB (noisy-trained) vs 29.81 dB (clean-trained); validation **31.74 vs 31.77 dB**; ~ 13 h on Tesla P100; 300 epochs.

Compute / model: Start with RED30, then switch to a **U-Net $\sim 10\times$ faster** with ≈ -0.2 dB hit.

Ablations worth trusting (why):

- **Correlated noise** slows convergence but final quality $\approx$ same (isolates the effect of gradient averaging).
- **Capture-budget** study shows benefit of collecting many noisy-noisy pairs vs few clean targets under fixed effort.
- **Loss choice for HDR/MC:** A relative HDR loss beats L2; applying non-linear tone maps to **targets** biases outputs dark.

Limitations (theirs & yours):

- Requires **independent** corruptions for input/target; non-zero-mean noise breaks the L2 expectation argument.
- Strong **inter-pixel correlation** increases gradient noise → slower training.
- Non-linear transforms on targets can skew the learned distribution (HDR caveat).

Repro status (notes): Code provided (NVlabs/noise2noise). Match their crop sizes, loss, noise sampling, and U-Net/RED30 configs.

Adoption plan:

- For photographic denoising, capture **two short-exposure shots** per scene → train with L2 on noisy-noisy pairs (ensure temporal independence / slight motion allowed but align if needed).
- For MRI, randomize k-space masks per pair (Bernoulli with retention parameter), train end-to-end with frequency-preservation term as described.
- Prefer U-Net for speed unless you need the last ~0.2 dB.

Open questions / follow-ups (inference):

- How far does this hold under **biased noise** or mild dependence between the two corruptions?
- What's the impact of **misalignment** and scene changes in handheld captures?
- Can we blend Noise2Noise with self-supervised single-image schemes for scenarios where pairs are hard to get?

TL;DR (3 bullets)

- **Idea:** Train on **noisy↔noisy** pairs; L2 learns the clean expectation, L1 the median.
- **Evidence:** Across three datasets and three noise types, **noisy-target PSNR** $\approx$ **clean-target PSNR**; MRI and Monte-Carlo sections back it up.
- **Caveats:** Need independent corruptions and careful loss/target handling (esp. HDR); correlated noise slows convergence.

If you'd like, I can turn this into a reusable Notion/PDF template and pre-fill it with these numbers.

